of the list, it will circle back and its successor will be the first vertex).

When it comes to the transitions to a new solution, we have a few options:

- Swapping adjacent vertices: after selecting a random position in the sequence, we would swap the vertex at the given index with its successor. For instance, the solution `[1,2,3,4]` could change into `[2,1,3,4]` or `[1,3,2,4]` etc...

  This transition corresponds to a local search, where only the immediate neighborhood of current solution is explored; this raises the probability that fine tuning leads to improvement, but makes leaps and getting out of local minima harder.

- Swapping any pair of vertices: both positions for the vertices to swap are chosen at random, and they are simply swapped: `[1,2,3,4]` can also become `[3,2,1,4]`, which was not allowed by the previous operator.

  These kind of transitions allow medium range search, although the two solutions are still quite close to each other.

- Generating a new sequence at random: this allow to leap across the whole domain at any stage of the algorithm, making it less likely to lead to fine-grain improvements towards a local minimum in the final stage of the simulation (because relatively few attempts will be made in the neighborhood of current solutions), but at the same time it will also leave the door open for